

MARS: Typed Executable Hypothesis Induction for Scientific Reasoning with Small Language Models

Anonymous submission

Abstract

Scientific reasoning benchmarks expose a failure mode that is easy to miss in standard agent evaluations: the difficult step is often not verifying a final answer, but constructing the right hypothesis family before the answer is known. This is especially problematic for small language models, whose extended reasoning traces can repeat shallow variants of the same wrong representation. We introduce MARS, a typed executable hypothesis-induction system that represents hypotheses as artifacts with typed holes, evidence contracts, validators, and residuals. A shared MARSCONTEXT routes generation through contract compilation, typed-hole closure, executable measurement, metamorphic validation, and residual-conditioned operator promotion. The model proposes local typed objects, while execution closes uncertain slots and stores failures as structured residuals rather than untyped bad answers. We evaluate the same kernel on three heterogeneous scientific reasoning benchmarks: DiscoveryBench, NewtonBench, and UltraHorizon. Complete-run evaluations with a GPT-4o-mini core reach 21.86 HMS on DiscoveryBench, 47.5% symbolic accuracy on NewtonBench, and 75.36 average score on UltraHorizon. Component ablations show larger gains on a stratified DiscoveryBench set when typed evidence contracts and answer-slot closure are enabled. These results do not imply that external scaffolding eliminates the need for strong model priors; rather, they show that making hypothesis construction typed, executable, and residual-aware turns many small-model failures into local, measurable objects. The central claim is structural: weak models reason more effectively when the hypothesis language is externalized and refined by evidence instead of being sampled only as free-form prose.

Introduction

Scientific discovery tasks ask an agent to do more than answer a question. The agent must decide what should be measured, what kind of relation is plausible, which variables play stable roles, and when a proposed explanation has been falsified. Recent benchmarks make this distinction explicit. DiscoveryBench casts data-driven discovery as hypothesis search over real datasets and metadata (Majumder et al. 2024). NewtonBench evaluates interactive scientific law discovery rather

than static curve fitting (Zheng et al. 2025). UltraHorizon stresses long-horizon partially observable exploration (Luo et al. 2025). Across these settings, the bottleneck is often not final verification but the construction of a useful hypothesis space before the answer is known.

This bottleneck is especially sharp for small language models. Standard inference-time scaffolds extend the trajectory: REACT interleaves reasoning and actions (Yao et al. 2023a), Reflexion and Self-Refine add verbal feedback (Shinn et al. 2023; Madaan et al. 2023), Tree-of-Thoughts and LATs search over reasoning branches (Yao et al. 2023b; Zhou et al. 2024), and CODEACT uses executable code as the action interface (Wang et al. 2024). These methods improve many agent tasks, but they still largely rely on the model to propose the intermediate concepts that make the search fruitful. If the model never represents the task in the right coordinates, more attempts can simply explore a larger set of wrong programs.

MARS starts from a different premise: hypothesis generation should be treated as an external, executable induction problem, not as a longer natural-language trace. A scientific hypothesis is represented as an artifact with typed holes, evidence contracts, executable measurements, validation tests, and residuals. The language model proposes local moves, but the environment closes uncertain parts whenever possible. For example, instead of asking the model to guess a complete law or discovery statement, the system asks which roles must be filled, which measurement can identify each role, which residual remains after execution, and which operator would make that residual simpler on future tasks.

The central object in MARS is a shared hypothesis state, MARSCONTEXT. It is read and written by heterogeneous components: a universal hypothesis kernel, evidence-contract compiler, estimand synthesizer, role canonicalizer, coordinate-probe engine, metamorphic verifier, and residual compiler. This is not a transcript memory. It is a typed state over hypotheses: what has been measured, which slots have been closed, which answer forms are admissible, which residuals recur, and which executable operators compress previous failures. In this respect MARS is closer in spirit to library-learning systems such as DreamCoder (Ellis et al. 2021) than to a pure agent loop, but its learned unit is not a domain-specific program alone. It is a reusable way of turning an underspecified scientific task into measurable hy-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

hypothesis objects. In ReAct-style agents, a failed tool result remains contextual feedback to the model; in MARS, it is mapped to a typed residual whose type constrains the next admissible executable operators.

The contribution is threefold. First, we formulate small-model scientific reasoning as *typed executable hypothesis induction*: a sequence of typed measurements, validations, and residual-conditioned language edits. Second, we instantiate this formulation in one shared kernel that spans tabular discovery, symbolic law induction, and long-horizon rule discovery without benchmark-specific solver modules. Third, we report full-run evaluations on three benchmark families and analyze where the shared mechanism succeeds or fails. The empirical results show that externalizing hypothesis generation makes failures local and diagnosable, while also revealing a clear remaining bottleneck: small models still need richer mechanisms for generating useful hypothesis representations when the missing object cannot be reduced to a measurement.

The paper tests a specific mechanism-level hypothesis: small models fail scientific reasoning tasks when intermediate hypotheses remain untyped strings, and they improve when those hypotheses become executable objects with typed slots, validators, and residuals. This makes the claim narrower than general “agent improvement” and broader than a benchmark solver: MARS is an interface for changing the hypothesis language available to the model at inference time.

Preliminaries

Task model. We consider a scientific reasoning task as an interface x and an answer space $\mathcal{A}$. The interface may expose a dataset with metadata, a simulator that can be probed, or a partially observable environment. The answer may be a natural-language hypothesis, a symbolic law, a transition program, or another executable scientific artifact. Each benchmark provides its own evaluator $S_x : \mathcal{A} \rightarrow \mathbb{R}$, but the system must operate before seeing the gold answer and cannot assume a benchmark-specific answer template.

Hypothesis families. Let $\mathcal{H}$ be a family of executable hypothesis artifacts. A hypothesis $h \in \mathcal{H}$ is not a string alone but a tuple

$$h = (z, \theta, m, v, r), \quad (1)$$

where z is a typed sketch, θ is a partial assignment to its holes, m is an executable measurement or transition program, v is a verifier, and r renders verified evidence into the benchmark answer format. This separates three operations that are often collapsed in an LLM prompt: choosing a form, measuring its slots, and writing the final answer.

Evidence contracts. Before proposing a final answer, MARS compiles the interface into an evidence contract

$$C(x) = \{s_i = (\tau_i, q_i, \nu_i)\}_{i=1}^k. \quad (2)$$

Here τ_i is the slot type, q_i is the observation or computation that could close the slot, and ν_i is a local validity predicate. A contract for tabular discovery may require population, outcome, predictor, direction, and statistic slots. A

contract for law induction may require coordinates, dimensionless groups, constants, and residual checks. A contract for long-horizon rule discovery may require state variables, transition clauses, and trace-level invariants.

Residuals and posterior scoring. Executing h on x produces evidence $E(h, x)$ and a typed residual

$$\rho(h, C) = C - \text{Cover}(E(h, x), C). \quad (3)$$

The kernel scores candidates by evidence fit, contract coverage, validation loss, and complexity:

$$\pi(h \mid x) \propto \exp\{\lambda_E E + \lambda_C \text{Cover} - \lambda_V V - \lambda_K K\}. \quad (4)$$

This resembles Bayesian model selection in spirit: a useful hypothesis should explain observations while remaining compact and stable under checks. The important operational point is that ρ is typed. A failure is not just a low score; it identifies a missing role, wrong coordinate system, invalid answer form, or unmodeled transition.

Table 1 makes the scoring parameterization explicit. The same decomposition is used across interfaces, while each operator normalizes its executable evidence to a comparable range before entering the shared posterior.

Table 1: Posterior terms used by the shared hypothesis kernel.

Term	Definition	Range	Weight
E	executable fit or local score	$[0, 1]$	24
Cover	contract slots covered	$[0, 1]$	34
V	metamorphic validation loss	$[0, 1]$	32
K	operator complexity	≥ 0	2

Evaluation metrics. DiscoveryBench reports Hypothesis Matching Score (HMS), a facet-based score over context, variables, and relations. NewtonBench reports symbolic accuracy (SA), RMSLE, and interactive discovery statistics across physical domains. UltraHorizon reports environment scores for long-horizon partially observable exploration, with trajectories designed to stress memory, planning, and tool use. Our experiments use these three benchmark families to test whether one hypothesis-state interface can span tabular discovery, symbolic law induction, and long-horizon rule induction.

Related Work

Reasoning-time scaffolds for LLM agents. Chain-of-thought and agentic prompting extend the computation performed at inference time. REACT couples reasoning traces with actions (Yao et al. 2023a); Reflexion and Self-Refine use feedback to revise later attempts (Shinn et al. 2023; Madaan et al. 2023); Tree-of-Thoughts and LATs make the search tree explicit (Yao et al. 2023b; Zhou et al. 2024); CODEACT unifies tool use around executable Python actions (Wang et al. 2024). These methods are closest to MARS at the level of execution, but they do not by themselves define what a scientific hypothesis object is. MARS uses the same broad agentic insight—interleave model calls with external computation—but makes the persistent unit a typed hypothesis state rather

than a reasoning transcript. This distinction matters when the model’s blind spot is representational. If a model does not propose the right variables, coordinates, answer form, or transition clauses, additional sampled traces can preserve the same missing object. MARS instead makes the missing object part of the state: a hole or residual that later operators can target.

Self-optimizing language-model pipelines. DSPy treats LM programs as optimizable computational graphs rather than hand-written prompt strings (Khatab et al. 2023). Self-Discover asks a model to compose reasoning structures before solving a task (Zhou et al. 2024). Voyager stores executable skills learned through interaction with Minecraft (Wang et al. 2023). These systems show that the right unit of adaptation may be a program, module, or reusable skill. MARS differs in two ways. First, its stored unit is tied to scientific evidence: contracts, measurements, residuals, and validators. Second, promotion is residual-conditioned; an operator is useful only if it closes typed holes or compresses recurrent failures across tasks.

Program induction and library learning. DreamCoder learns a domain language and neural search policy through a wake-sleep loop over solved and imagined programs (Ellis et al. 2021). Later library-learning work further explores how abstractions reduce search breadth and depth. This line is a major conceptual ancestor of MARS: both systems treat expertise as a changing language rather than a fixed prompt. The difference is that MARS operates in mixed scientific interfaces where the answer may be prose, a law, or a transition program, and where holes can often be closed by measurements rather than by full program synthesis.

Symbolic regression and scientific law discovery. Symbolic regression recovers compact formulas from data. AI-Feynman exploits physics-inspired structure such as symmetry and separability (Udrescu and Tegmark 2020); PySR provides a practical evolutionary symbolic-regression engine for scientific modeling (Cranmer 2023). NewtonBench moves beyond static formula fitting by requiring interactive rediscovery of shifted physical laws (Zheng et al. 2025). MARS borrows the insistence on executable hypotheses, but does not assume that every task reduces to equation recovery. Its coordinate probes are one operator family inside a broader hypothesis-state kernel.

Causal and experimental-design perspectives. Scientific hypotheses are not merely predictors; they make claims about stable relations under interventions, environments, or perturbations. Structural causal models formalize this distinction (Pearl 2009), invariant prediction uses stability across environments to identify causal structure (Peters, Buhlmann, and Meinshausen 2016), and Bayesian experimental design studies how to choose measurements that maximize information (Rainforth et al. 2023). MARS is not a causal discovery algorithm, but it adopts the same discipline: a hypothesis must state what evidence would support it, what transformation should preserve it, and what residual would falsify or refine it.

Benchmarks for scientific agents. DiscoveryBench, NewtonBench, and UltraHorizon expose different faces of the same problem: the agent must build an internal research object before it can answer. DiscoveryBench emphasizes data and metadata alignment (Majumder et al. 2024); NewtonBench emphasizes interactive law induction (Zheng et al. 2025); UltraHorizon emphasizes long-horizon memory and partial observability (Luo et al. 2025). We use these benchmarks not as separate engineering targets but as probes of whether one hypothesis-state representation can support multiple scientific task interfaces.

MARS

Overview

MARS is a hypothesis-induction kernel around a small language model. The model is used to propose typed sketches, measurements, and repairs, but the state of the search is external and executable. The same loop is used for all benchmarks: compile a contract, generate artifacts, execute measurements, validate them, store residuals, and occasionally promote a residual pattern into a new operator. Operators do not pass only text to later calls; they update one typed context containing contracts, artifacts, evidence, residuals, and promoted operators.

The design is governed by three invariants. *Typed state*: every useful intermediate claim must be represented as a slot, artifact, validator, or residual rather than only as prose. *Executable closure*: whenever a slot can be identified by a computation or probe, the system closes it by execution instead of asking the model to guess it again. *Residual discipline*: a failed hypothesis can expand the operator language only when the residual type recurs and the proposed operator improves held-out tasks after a complexity penalty. These invariants are the difference between MARS and a longer self-refinement transcript.

The MARSCONTEXT

For a task x , the context is

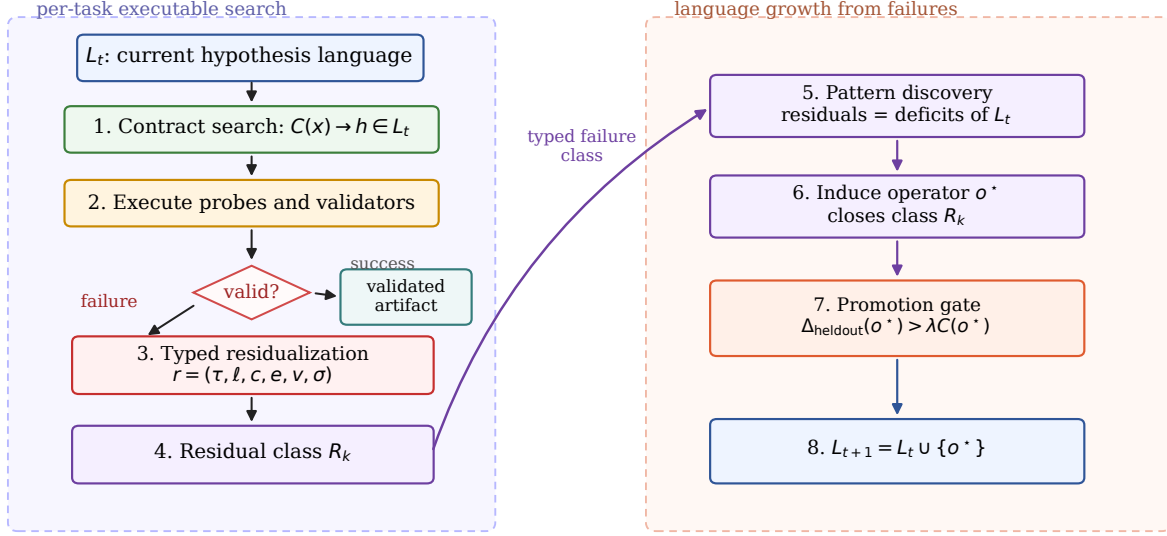
$$\mathcal{M}_x = (C, \mathcal{H}, \Theta, \mathcal{E}, \mathcal{R}, \mathcal{O}, h^*), \quad (5)$$

where C is the evidence contract, $\mathcal{H}$ is the pool of candidate artifacts, Θ stores closed typed holes, $\mathcal{E}$ stores executable evidence, $\mathcal{R}$ stores typed residuals, $\mathcal{O}$ is the active operator set, and h^* is the current best answer artifact. All operators read and write this object. This is the main architectural constraint: every stage must return a state update, not a private chain of thought.

Each operator $O_j \in \mathcal{O}$ has the same interface,

$$O_j(\mathcal{M}_x, x) \rightarrow \{(h, e, \rho, \Delta\mathcal{M})\}, \quad (6)$$

where h is a proposed artifact, e is executed evidence, ρ is the typed residual, and $\Delta\mathcal{M}$ is the state update. This interface is what makes the system benchmark-independent. A tabular estimand, a symbolic coordinate probe, and a transition-rule inducer differ internally, but the kernel only sees artifact, evidence, residual, and update.



Novelty: operators are induced from typed validation residuals, not from successful-program compression or generic text feedback.

Figure 1: Residual-guided hypothesis-language induction. The per-task loop searches for executable typed hypotheses in the current language L_t and turns validation failures into typed residual classes. The language-growth loop interprets recurring residual classes as expressivity deficits, induces typed operators that close those classes, and admits them only when they improve held-out search beyond an explicit complexity cost.

End-to-end trace. In a NewtonBench-style trajectory, the model does not guess the law directly: the contract asks for coordinate, exponent, constant, and residual slots; a sketch $y = c\phi(x)^p$ is executed; residual curvature activates a coordinate probe; the probe closes the coordinate, exponent, and constant slots; and the held-out validator accepts the artifact. The failed first probe changes typed state rather than merely changing the next prompt.

Operators

Contract compiler. The contract compiler maps the input interface to required slots and local validators. It does not predict the gold answer. It predicts what any valid answer must make observable: answer type, scope, variables, measurement target, relation form, and rendering constraints.

Typed-hole closure. Instead of asking the model for a complete hypothesis, MARS asks for sketches with typed holes and then closes the holes by small executable probes whenever possible. For a tabular question, a hole may be an outcome role or a window operator. For a law-discovery task, it may be a coordinate chart or exponent. For a sequence task, it may be a state variable or transition clause. Closing a hole adds an assignment to Θ and reduces the search space for later holes.

Executable estimands and coordinate probes. The estimand operator creates statistical objects for tabular discovery: group contrasts, interaction effects, coefficient shifts, multi-item proportions, and time-window measurements.

The coordinate-probe operator creates compact transformations for laws and rules: log-log fits, affine and polynomial lifts, ratios, finite differences, thresholds, and simple state-transition clauses. Both operators produce evidence objects rather than only prose rationales.

Metamorphic verifier. The verifier checks whether the artifact answers the requested question under transformations that should preserve the object: renaming irrelevant columns, holding answer form fixed, changing display order, perturbing nuisance values, or replaying held-out traces. These checks are label-free and therefore available before benchmark evaluation.

Induction Algorithm

The scoring step uses the posterior-style objective from the preliminaries:

$$\text{score}(h) = \lambda_E E(h, x) + \lambda_C \text{cover}(h, C) - \lambda_V V(h, C) - \lambda_K K(h). \quad (7)$$

Here V is metamorphic validation loss and K is an artifact complexity penalty. The same score ranks natural-language discovery hypotheses, symbolic laws, and transition programs.

Residual-Conditioned Operator Promotion

The residual-extension step is not arbitrary code growth. A residual cluster $R = \{\rho_1, \dots, \rho_m\}$ can propose an operator only if it identifies a repeatable missing transformation.

Algorithm 1 MARS typed executable hypothesis induction

Require: task interface x , operator set $\mathcal{O}_0$, budget B

```

1:  $C \leftarrow \text{COMPILECONTRACT}(x)$ 
2:  $\mathcal{M} \leftarrow (C, \emptyset, \emptyset, \emptyset, \emptyset, \mathcal{O}_0, \emptyset)$ 
3: for  $t = 1$  to  $B$  do
4:    $\mathcal{P} \leftarrow \emptyset$ 
5:   for all  $O_j \in \mathcal{M}.\mathcal{O}$  do
6:      $\mathcal{P} \leftarrow \mathcal{P} \cup O_j(\mathcal{M}, x)$ 
7:   end for
8:   for all  $(h, e, \rho, \Delta\mathcal{M}) \in \mathcal{P}$  do
9:     update  $\mathcal{M}$  with  $h, e, \rho, \Delta\mathcal{M}$ 
10:    score  $h$  by evidence, coverage, validation, and complexity
11:    if  $h$  improves  $h^*$  then
12:       $h^* \leftarrow h$ 
13:    end if
14:  end for
15:  if  $h^*$  satisfies  $C$  and validation checks then
16:    return  $\text{RENDER}(h^*, C)$ 
17:  end if
18:   $\mathcal{O}_{new} \leftarrow \text{COMPIRESIDUALS}(\mathcal{M}, \mathcal{R})$ 
19:  promote operators in  $\mathcal{O}_{new}$  that pass cross-task checks
20: end for
21: return  $\text{RENDER}(h^*, C)$ 

```

Formally, a candidate operator O' is promoted when

$$\Delta(O') = \mathbb{E}_{x \sim \mathcal{D}_{held}} [S(h_{O'}^*(x)) - S(h^*(x))] - \beta K(O') > 0, \quad (8)$$

and the improvement is not confined to the task that produced the residual. This rule is meant to prevent an ever-growing bag of benchmark-specific tricks: an operator must improve held-out tasks under the same contract and scoring interface while paying a complexity penalty.

Why This Helps Small Models

MARS changes the role of the language model. The model does not need to hold the entire long-horizon proof in context or invent the final hypothesis in one sample. It repeatedly proposes local typed objects whose consequences can be executed. Each closed hole constrains later holes, each verifier rejects an invalid answer form before final scoring, and each residual becomes a structured training signal for the outer loop. The intended gain is therefore not more verbal reasoning, but a better external search geometry for weak models.

Experimental Evaluation

Benchmarks and protocol

We evaluate on three benchmark families that stress different scientific interfaces while sharing the same central difficulty: the agent must construct a hypothesis object before it can answer. DiscoveryBench provides real datasets, metadata, and natural-language discovery goals; the final answer is scored by Hypothesis Matching Score (HMS) and a consistency-HMS variant. NewtonBench provides interactive scientific-law discovery tasks in shifted physical worlds; we report symbolic accuracy over all tasks (SA-all) and over

non-abstained answers (SA-answered). UltraHorizon provides long-horizon partially observable rule-induction environments; we report the reproduced paper-style environment score.

The reported system uses a GPT-4o-mini core throughout. We use one shared hypothesis-induction stack: contract compilation, typed-hole closure, estimand synthesis, coordinate probing, metamorphic validation, and residual-conditioned promotion. Benchmark adapters expose observations and accept final artifacts, but the reported system does not include separate benchmark-specific solver modules. We exclude diagnostic ceiling runs that used task-family measurement bootstraps or hand-engineered rendering fixes.

Reproducibility audit. For DiscoveryBench we evaluate the repository real-test task rows using the benchmark evaluator, producing 239 predictions and 239 scored rows. For NewtonBench we evaluate the full cross-product of 12 physics domains, three difficulty levels, three law versions, and three system variants, for 324 symbolic-law tasks. For UltraHorizon we evaluate 32 seeds for each of the three official environments, for 96 total runs, using the paper-style judge, hard difficulty, 50-step episodes, and no environment hints or measurement bootstraps. The row-level outputs are retained for each benchmark, and the main table is generated only from complete runs with the expected row counts.

Table 2: Benchmark interfaces and metrics.

Benchmark	Interface	Artifact	Metric
DiscoveryBench	tables, metadata, goal	hypothesis	HMS / Cons-HMS
NewtonBench	simulator, observations	symbolic law	SA-all / SA-answered
UltraHorizon	partial-observation env.	transition program	score

Main full-run results

Table 3 gives the current full-run results. Each row is a complete benchmark run under the corresponding native metric. The pattern is informative rather than uniformly dominant. On DiscoveryBench, MARS improves over published non-oracle ReAct-style references, while still trailing the GPT-4o MARS comparison. On NewtonBench, MARS reaches 47.5% SA-all, within 0.3 points of the published o4-mini reference and below stronger frontier-model agents. On UltraHorizon, the reproduced paper-style run is strong for the compiler-enabled MARS system, but the per-environment breakdown shows that grid exploration remains substantially harder than sequence and bio rule induction.

We therefore make two claims from the table. The first is empirical: the same GPT-4o-mini-core hypothesis kernel produces nontrivial full-run results across three different scientific interfaces. The second is diagnostic: the failures concentrate in different residual types across benchmarks, which supports the use of a shared typed state rather than benchmark-specific prompt repair. We do not claim that the current operator set dominates all larger-model systems.

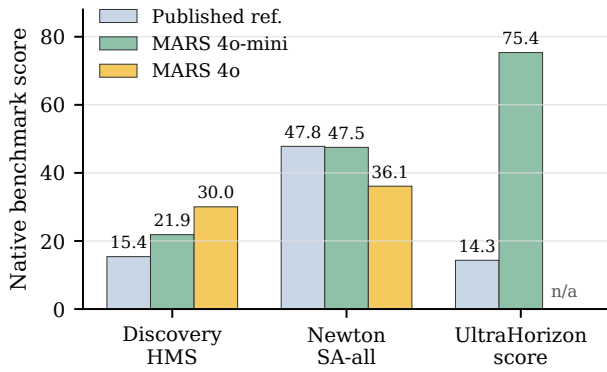


Figure 2: Headline comparison under each benchmark’s native metric. Reference rows are contextual rather than controlled ablations because the published systems use different models, budgets, and interfaces.

Table 3: Full-run GPT-4o-mini-core MARS results.

Benchmark	<i>N</i>	Primary	Secondary / breakdown
DiscoveryBench	239	21.86 HMS	28.14 Cons-HMS
NewtonBench	324	47.5% SA-all	64.7% SA-answered
UltraHorizon	96	75.36 score	compiler-enabled; Grid 54.38, Seq 81.25, Bio 90.47

Table 4: Published references under each benchmark’s native metric. Reference systems are not all matched by model, budget, or protocol, so they should be read as context rather than controlled ablations.

Benchmark	System	Core	Reported score
DiscoveryBench	GPT-4o ReAct	GPT-4o	15.4 HMS
DiscoveryBench	CodeGen	GPT-4-preview	16.3 HMS
DiscoveryBench	Reflexion	GPT-4 family, oracle feedback	24.5 HMS
DiscoveryBench	MARS	GPT-4o-mini	21.86 HMS
NewtonBench	Vanilla agent	GPT-5	75.9% SA
NewtonBench	Vanilla agent	Gemini-2.5-Pro	65.4% SA
NewtonBench	Vanilla agent	o4-mini	47.8% SA
NewtonBench	MARS	GPT-4o-mini	47.5% SA-all
UltraHorizon	Best free-step LLM	reported best LLM	14.33 score
UltraHorizon	Human	human	26.52 score
UltraHorizon	MARS, compiler-enabled	GPT-4o-mini	75.36 score

Detailed component ablations and diagnostic deltas are reported after the references. We place them outside the main narrative because their role is to attribute failures and component contributions, while the primary claims in Table 3 are based on complete benchmark runs.

What the full runs show

DiscoveryBench. The DiscoveryBench result supports the main hypothesis-state claim most directly, but also gives the clearest warning against overclaiming. Natural-language discovery questions are difficult for small models because the model must infer the relevant table, population, variables, and relation before writing the hypothesis. MARS replaces much of this latent choice with evidence contracts and executable estimands. The complete 239-task run reaches 21.86 HMS, while the controlled component set rises from 8.33

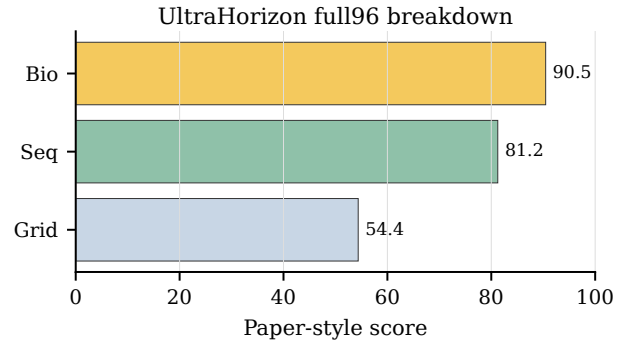


Figure 3: UltraHorizon environment breakdown for the complete 96-run compiler-enabled evaluation. The grid environment remains the dominant long-horizon bottleneck.

HMS with typed modules disabled to 42.27 HMS with the full stack. The remaining gap is not primarily arithmetic: errors often occur when the system measures a reasonable relation but renders it in a facet that the benchmark does not credit, or when multiple files must be assembled into a single scientific object.

NewtonBench. NewtonBench exposes the limit of external scaffolding. The full run answers 238 of 324 tasks and reaches 64.7% symbolic accuracy on answered tasks, but abstentions lower SA-all to 47.5%. This is close to the published o4-mini reference but below larger-model baselines. The failure mode is therefore specific: when a useful coordinate chart or law family is proposed, executable checking is effective; when no active operator proposes the right representation, the system abstains or fails compilation. The result argues for richer representation generation, not for more prompt-level retrying.

UltraHorizon. UltraHorizon shows the strongest absolute score under the reproduced paper-style judge, but also the clearest internal heterogeneity. The reported score is for the compiler-enabled MARS system: the post-budget evidence compiler can turn long tool traces into a terminal artifact when the agent has not committed. Strict no-fallback checks are substantially weaker, as reported in the appendix. Grid tasks remain the main bottleneck, with many zero-score runs caused by insufficient exploration or wrong state abstraction. This is important for the paper claim: the result is a strong long-horizon score for the full MARS pipeline, but not a solved benchmark and not a raw-agent ceiling.

Residual analysis

The distinctive mechanism in MARS is that failed hypotheses are stored as typed residuals. A residual can say that a role is missing, a coordinate system is wrong, a renderer is incompatible with the answer form, or a transition clause fails held-out replay. This changes the search problem: the next operator is selected because it can close a residual type, not because a language model sampled a longer chain of thought.

Across the three full runs, residuals have different practical meanings. In DiscoveryBench, residuals are mostly semantic and structural: wrong table, wrong population, wrong denominator, or wrong answer facet. In NewtonBench, residuals are algebraic: missing coordinate transforms, constants, exponents, and unit-compatible charts. In UltraHorizon, residuals are temporal: transition clauses fail on replay, an invariant holds only locally, or an exploration policy does not expose the evidence needed to identify the rule. These residual types are the basis for operator promotion. An operator is kept only if it improves held-out tasks after paying a complexity penalty, which is the mechanism used to limit benchmark-specific growth.

Failure analysis

The remaining errors fall into four recurring classes. *Representation misses* occur when no active operator proposes the right hypothesis family. These dominate NewtonBench abstentions and some UltraHorizon grid failures. *Renderer misses* occur when the system measures useful evidence but expresses it in an answer form that does not match the evaluator. These are common in DiscoveryBench. *Assembly misses* occur when the task requires joining evidence across multiple files, time windows, or schemas. These currently limit DiscoveryBench. *Exploration misses* occur when the system cannot collect the observations needed to identify a long-horizon rule within the action budget; these dominate the UltraHorizon grid gap.

This failure analysis is part of the practical value of MARS. In a transcript-only agent, failures are often just bad final answers. In MARS, the failure is attached to the missing typed object: a role, a coordinate, a validator, a renderer, or an exploration operator. That makes later improvement measurable without changing the benchmark-specific code path.

Limitations

MARS is a hypothesis-induction architecture for improving small-model scientific reasoning through external typed state and executable evidence. Its main limitation is operator coverage: the system improves hypothesis search when the missing object can be represented as a contract, a typed hole, a validator, a renderer, or a residual-induced operator. When the right representation is absent, ranking and verification cannot recover it by themselves.

The three benchmark runs expose this limitation in different ways. DiscoveryBench still depends on semantic rendering and multi-file assembly. NewtonBench is limited by missing coordinate charts and abstentions: the system is accurate on many answered tasks, but it does not yet cover enough law families to surpass stronger published agents. UltraHorizon shows strong sequence and bio rule induction, but grid tasks reveal an exploration problem: without the right state abstraction, the agent may never collect the evidence needed to induce the rule.

The residual-operator mechanism is therefore a disciplined form of adaptation, not a proof of unbounded autonomous self-improvement. It demonstrates that typed failures can be

converted into reusable operators under held-out checks and complexity penalties. A stronger version must learn broader operator families from residuals while preserving the same contract, validation, and complexity controls.

Conclusion

We presented MARS, a typed executable hypothesis-induction system for scientific reasoning with small language models. The main claim is structural: weak models benefit when hypothesis generation is moved from free-form prose into an external state of typed artifacts, executable measurements, validators, and residuals. Across data-driven discovery, symbolic law induction, and long-horizon rule induction, one GPT-4o-mini-core kernel obtains competitive full-run results on three benchmark families while exposing clear residual types when it fails. The experiments identify where external structure helps most: when the system can decompose a hypothesis into measurable slots, execute probes to close them, validate the resulting artifact, and promote recurring residuals into better operators. This suggests a concrete path for small-model scientific agents: improve the hypothesis language itself, guided by evidence and typed residuals, rather than only sampling more reasoning traces from a fixed language.

References

- Bodhisattwa Prasad Majumder, Harshit Surana, Dhruv Agarwal, Bhavana Dalvi Mishra, Abhijeetsingh Meena, Aryan Prakhar, Tirth Vora, Tushar Khot, Ashish Sabharwal, and Peter Clark. DiscoveryBench: Towards data-driven discovery with large language models. arXiv preprint arXiv:2407.01725, 2024.
- Tianshi Zheng, Kelvin Kiu-Wai Tam, Newt Hue-Nam K. Nguyen, Baixuan Xu, Zhaowei Wang, Jiayang Cheng, Hong Ting Tsang, Weiqi Wang, Jiabin Bai, Tianqing Fang, Yangqiu Song, Ginny Y. Wong, and Simon See. NewtonBench: Benchmarking generalizable scientific law discovery in LLM agents. arXiv preprint arXiv:2510.07172, 2025.
- Haotian Luo, Huaisong Zhang, Xuelin Zhang, Haoyu Wang, Zeyu Qin, Wenjie Lu, Guozheng Ma, Haiying He, Yingsha Xie, Qiyang Zhou, Zixuan Hu, Hongze Mi, Yibo Wang, Naiqiang Tan, Hong Chen, Yi R. Fung, Chun Yuan, and Li Shen. UltraHorizon: Benchmarking agent capabilities in ultra long-horizon scenarios. arXiv preprint arXiv:2509.21766, 2025.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *ICML*, 2024.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *ICML*, 2024.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. arXiv preprint arXiv:2310.03714, 2023.

Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V. Le, Ed H. Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. Self-Discover: Large language models self-compose reasoning structures. arXiv preprint arXiv:2402.03620, 2024.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023.

Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sable-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning. In *PLDI*, 2021.

Silviu-Marian Udrescu and Max Tegmark. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16), 2020.

Miles Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. arXiv preprint arXiv:2305.01582, 2023.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

Jonas Peters, Peter Buhlmann, and Nicolai Meinshausen. Causal inference by using invariant prediction: Identification and confidence intervals. *Journal of the Royal Statistical Society: Series B*, 78(5):947–1012, 2016.

Tom Rainforth, Adam Foster, Desi R. Ivanova, and Freddie Bickford Smith. Modern Bayesian experimental design. arXiv preprint arXiv:2302.14545, 2023.

Appendix: Detailed Ablations

This appendix reports component ablations used to diagnose the current MARS implementation. The goal is attribution: which parts of the universal hypothesis-induction stack change the measured behavior, and which parts are not currently bottlenecks. These runs use the same GPT-4o-mini core as the main experiments and the native evaluators for each benchmark family.

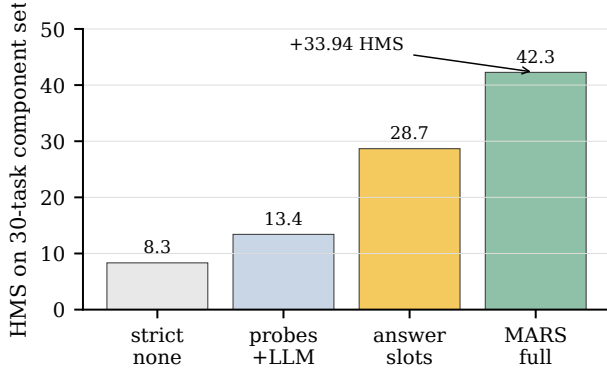


Figure 4: DiscoveryBench component attribution on the fixed 30-task component set. The strict-none row disables typed Discovery modules; the full stack adds answer-slot closure, evidence contracts, executable measurements, validation, and residual-conditioned rendering.

Table 5: Component ablations. DiscoveryBench uses a fixed stratified task set for component attribution, NewtonBench uses the full 324-task grid, and UltraHorizon reports grid and sequence component ablations with the same paper-style judge used in the full run.

Benchmark	Condition	<i>N</i>	Score
DiscoveryBench	strict none	30	8.33 HMS
DiscoveryBench	probes + LLM renderer	30	13.41 HMS
DiscoveryBench	hypothesis workbench	30	12.11 HMS
DiscoveryBench	slot contract only	30	11.00 HMS
DiscoveryBench	answer-slot stack	30	28.67 HMS
DiscoveryBench	MARS full	30	42.27 HMS / 52.27 Cons-HMS
NewtonBench	operator-chart enabled	324	46.0% SA-all / 62.3% SA-answered
NewtonBench	operator-chart disabled	324	47.5% SA-all / 64.7% SA-answered
UltraHorizon	MARS full, grid+seq	8	95.0 score
UltraHorizon	no reflector, compiler on, grid+seq	8	95.0 score
UltraHorizon	no refl./memory/futility, compiler on, grid+seq	8	97.5 score

Table 6: Diagnostic deltas derived from Table 5. Positive values indicate that the second condition scores higher than the first under the listed metric.

Benchmark	Comparison	Delta
DiscoveryBench	probes + LLM renderer → MARS full	+28.86 HMS
DiscoveryBench	strict none → MARS full	+33.94 HMS
DiscoveryBench	slot contract only → answer-slot stack	+17.67 HMS
DiscoveryBench	answer-slot stack → MARS full	+13.60 HMS
NewtonBench	chart enabled → chart disabled	+1.50 SA-all
NewtonBench	chart enabled → chart disabled	+2.40 SA-answered
UltraHorizon	MARS full → no reflector, compiler on	+0.00 score
UltraHorizon	MARS full → no refl./memory/futility, compiler on	+2.50 score

Table 7: UltraHorizon strict sanity checks. These rows separate the compiler-enabled MARS pipeline from raw or no-fallback agent execution.

Condition	Env./seeds	<i>N</i>	Score
Raw GPT-4o-mini, no hints, no fallback	grid+seq+bio, seed 0	3	0.0 mean
Strict MARS, no fallback/compiler	bio, seed 0	1	0.0 score
MARS, compiler-enabled	bio, seeds 0–3	4	100.0 mean
MARS, compiler-enabled	grid+seq+bio, seeds 0–3	12	96.67 mean

The DiscoveryBench ablation is the clearest positive component test: single-stage probes, the hypothesis workbench, and slot contracts alone are insufficient, while answer-slot closure and the full evidence-contract stack produce large gains. The NewtonBench ablation has the opposite diagnostic meaning. Disabling residual-born operator charts slightly improves the full grid, so the reported Newton result should not be attributed to a benchmark-specific chart library. Instead, it is carried by the shared active-probe, residual-gating, and symbolic-verification kernel. The UltraHorizon grid+sequence rows should be read as component toggles rather than no-module baselines: the final evidence compiler remains enabled in these conditions. Their non-monotone behavior therefore indicates that reflector, memory selection, and futility detection are not the current bottleneck for that component slice, not that a raw weak agent solves the environment. The benchmark-level UltraHorizon claim remains the complete 96-run result reported in Table 3. Table 7 makes this separation explicit: without the final compiler/fallback path, the same weak core does not solve the bio task in the audited seed.